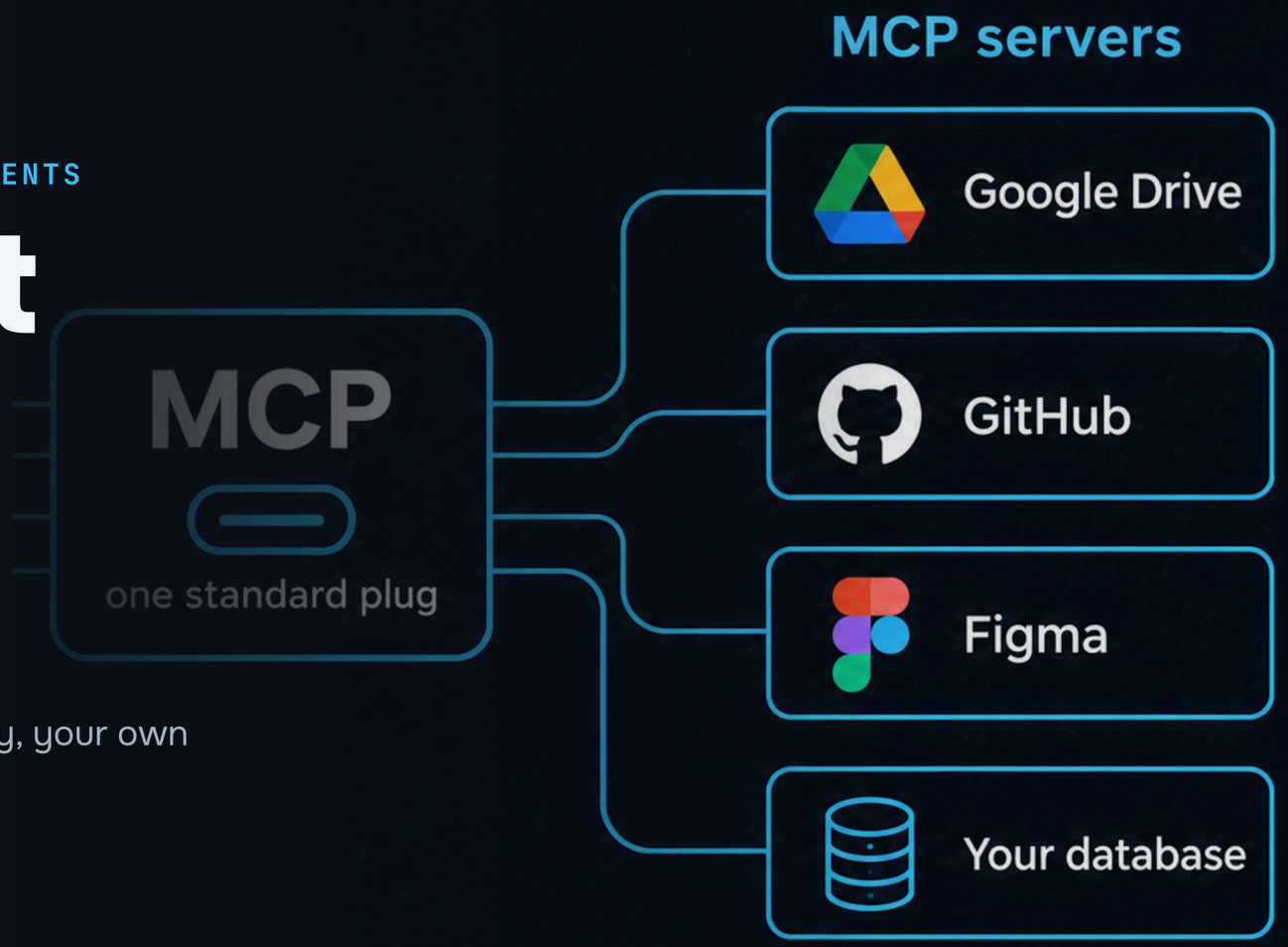
GENAI BASICS • LESSON 12 • MODULE 3: MODELS & AGENTS

MCP and agent frameworks

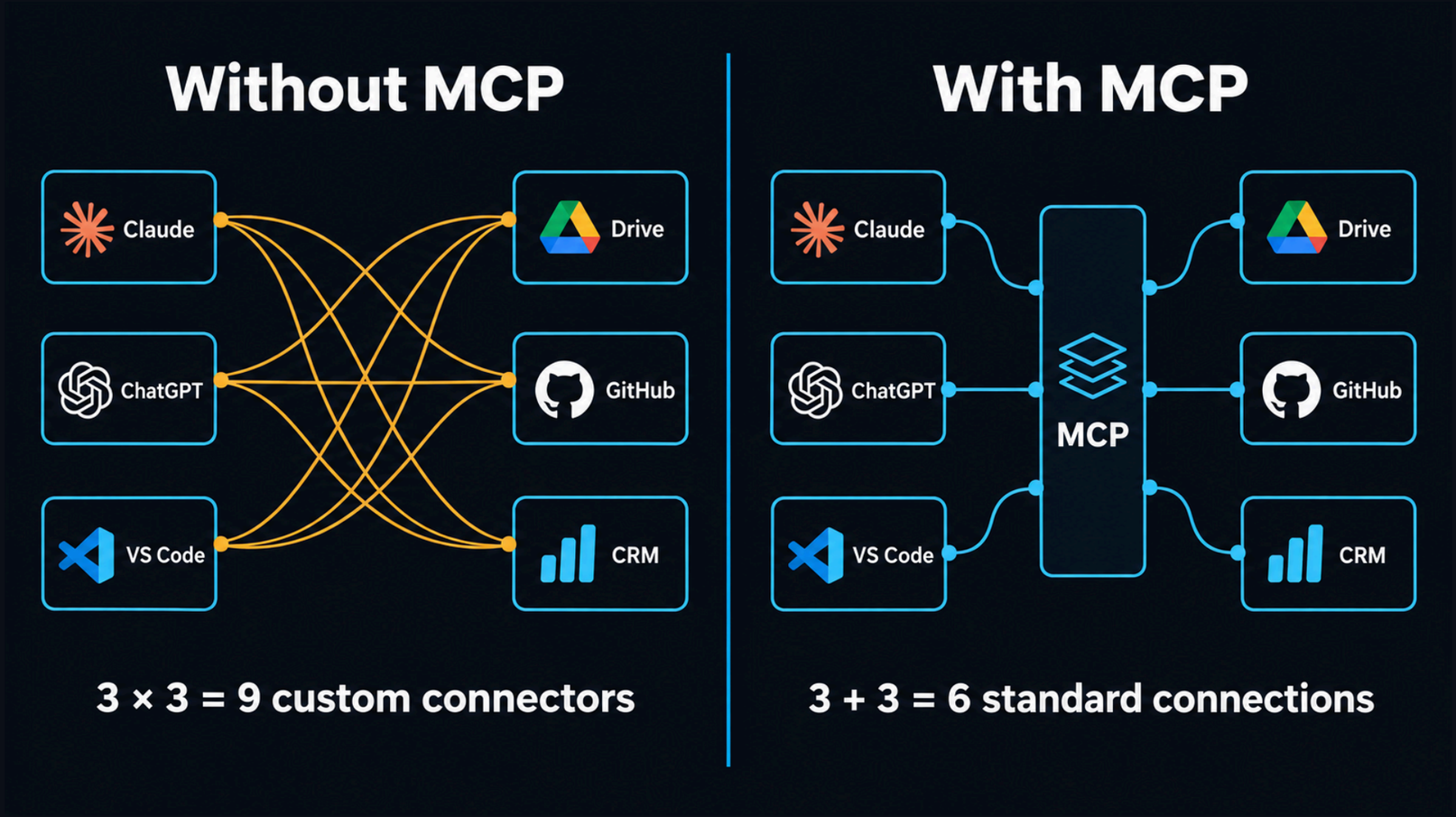
One standard “plug” to connect tools to any AI app

Hosts, servers, Claude connectors, the MCP registry, security, your own mini server, agent frameworks. Data as of October 2026.



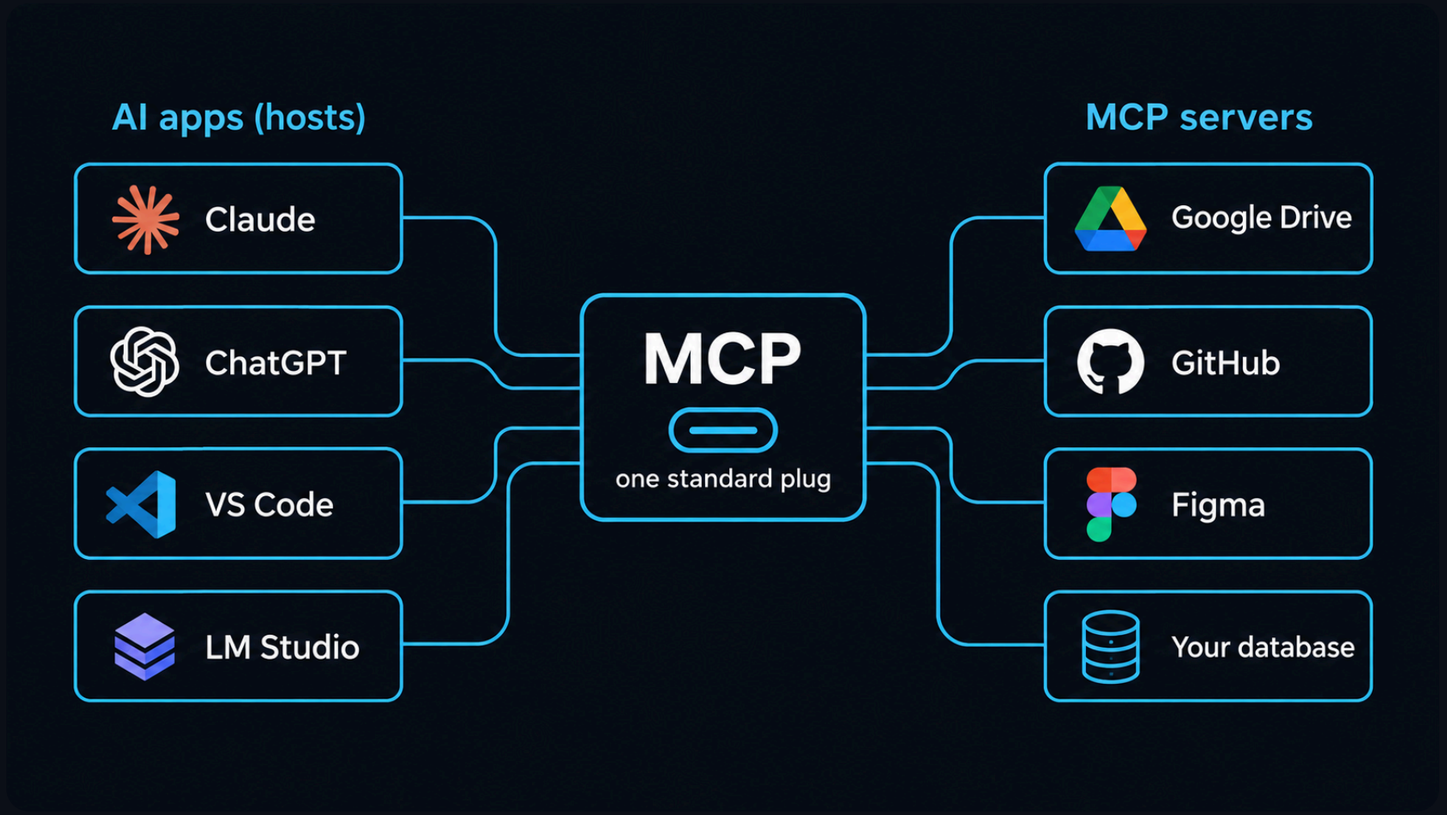
THE PROBLEM

Without a standard, every AI app is wired to every service separately



In L11 we described a function for one model by hand. Ten apps × ten services = a hundred such descriptions. MCP turns that into 10 + 10.

MCP is like USB-C for AI apps: one plug for any tool



MCP (Model Context Protocol) is an open standard for connecting AI apps to data and tools. Created by Anthropic in November 2024.

A service ships **one MCP server**, and it works in Claude, ChatGPT, VS Code, LM Studio.

The USB-C comparison comes from the official docs at modelcontextprotocol.io

HOW IT WORKS

Three roles: the host app, a client inside it and a server with tools

Host
the AI app you work in
Claude, ChatGPT, VS Code

→

Client
the “socket” inside the host, one per server
built into the app

→

Server
a program that offers tools and data
Google Drive, GitHub, Figma

WHAT A SERVER OFFERS THE HOST

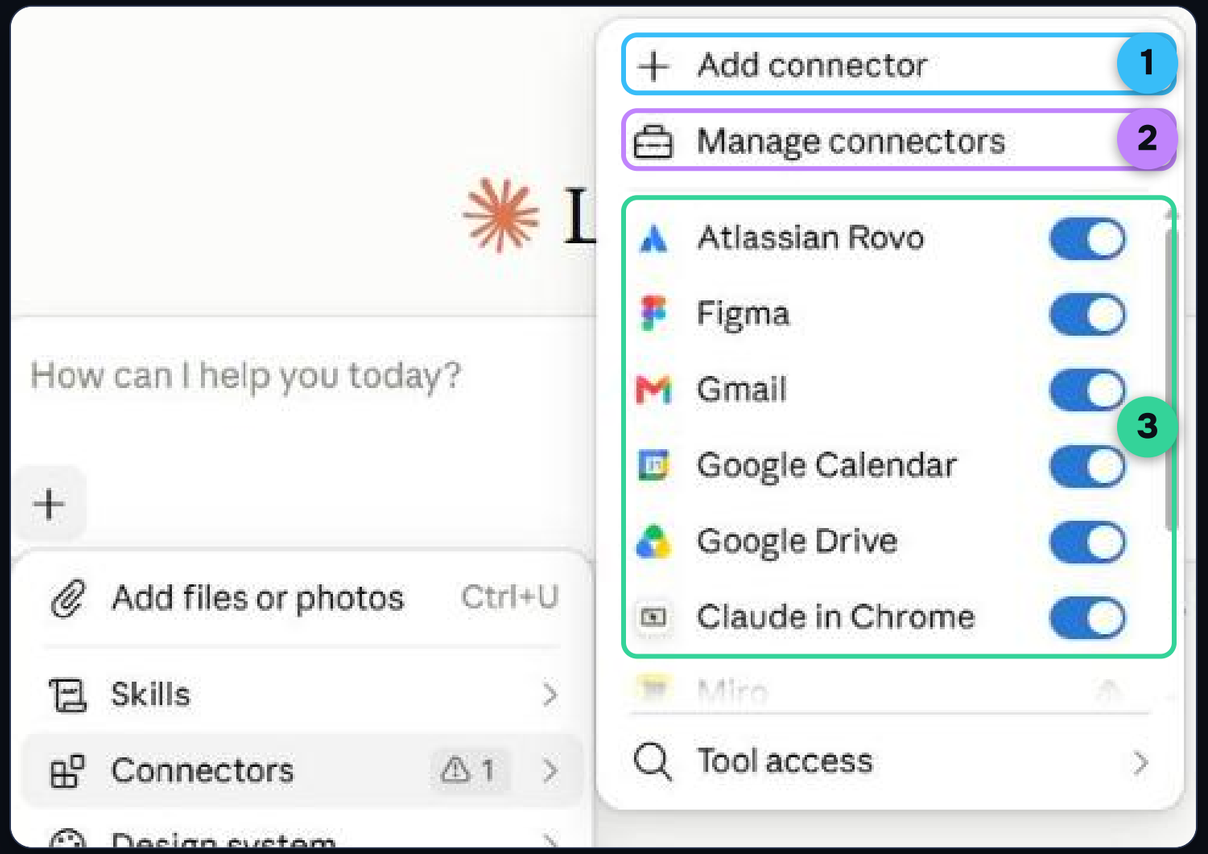
1. **Tools**: actions: find an order, create a task
2. **Resources**: data to read: files, records
3. **Prompts**: ready-made request templates

> As a user you need two words: the **app** (host) and the **connection** (server). The client works out of sight.

YOU HAVE SEEN THIS

Connectors in Claude are MCP servers: switch on and use

CLAUDE.AI → + → CONNECTORS · CAPTURED FOR L09



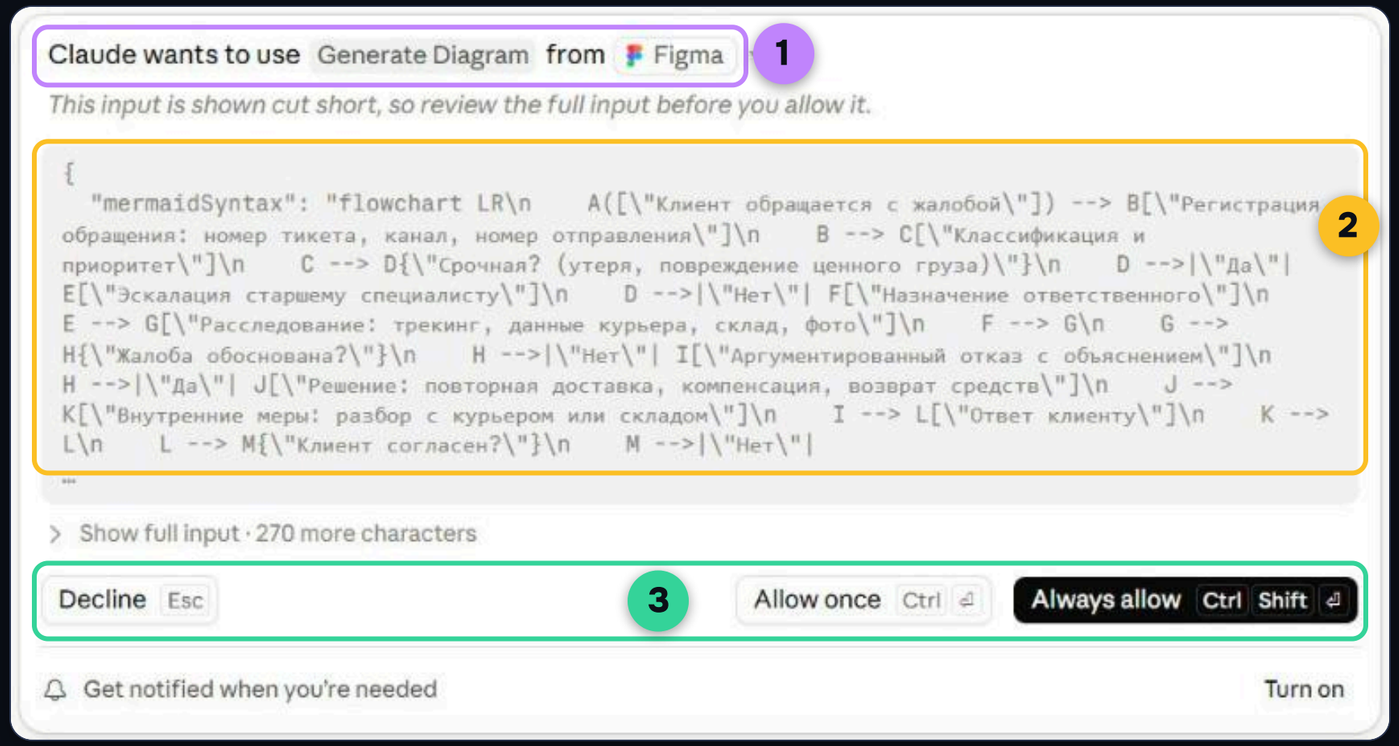
1. **Add connector**: connect a new server, including your own by URL
2. **Manage connectors**: which tools are allowed
3. **Connected**: Atlassian, Figma, Gmail, Calendar, Drive, Chrome — each can be switched off

> Switch off connectors you do not need in this chat: fewer tools, fewer mistakes and risks.

PERMISSIONS

Before a tool call Claude asks permission and shows what it will send

CLAUDE.AI · FIGMA TOOL “GENERATE DIAGRAM” · CAPTURED FOR L09



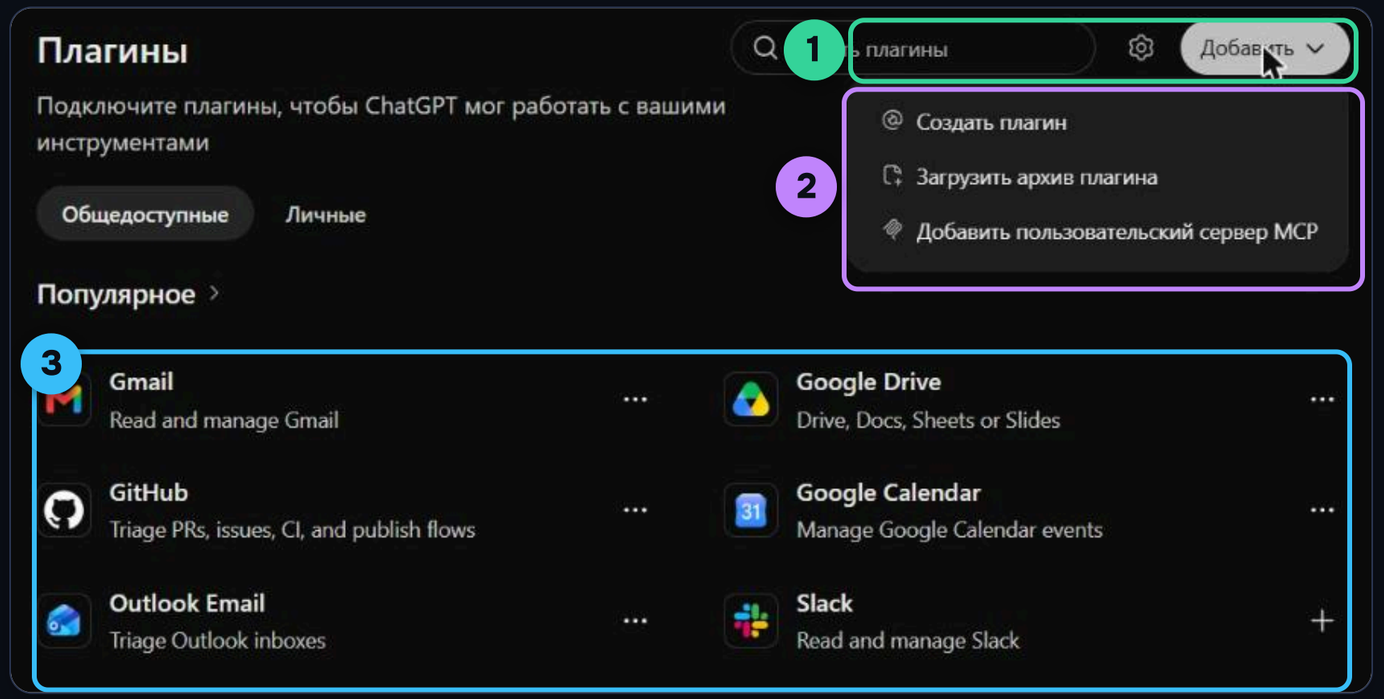
1. **Which** tool from **which** server
2. **Exactly what** goes to the service: read it; if cut short, open Show full input
3. **Decline · Allow once · Always allow**

> “Always allow” only for safe read-only tools. For sending, deleting, paying — approve every time.

WHERE IT WORKS

In ChatGPT the same thing is called Plugins — and you can add your own MCP server too

CHATGPT.COM → PLUGINS → ADD · CAPTURED 16.10.2026



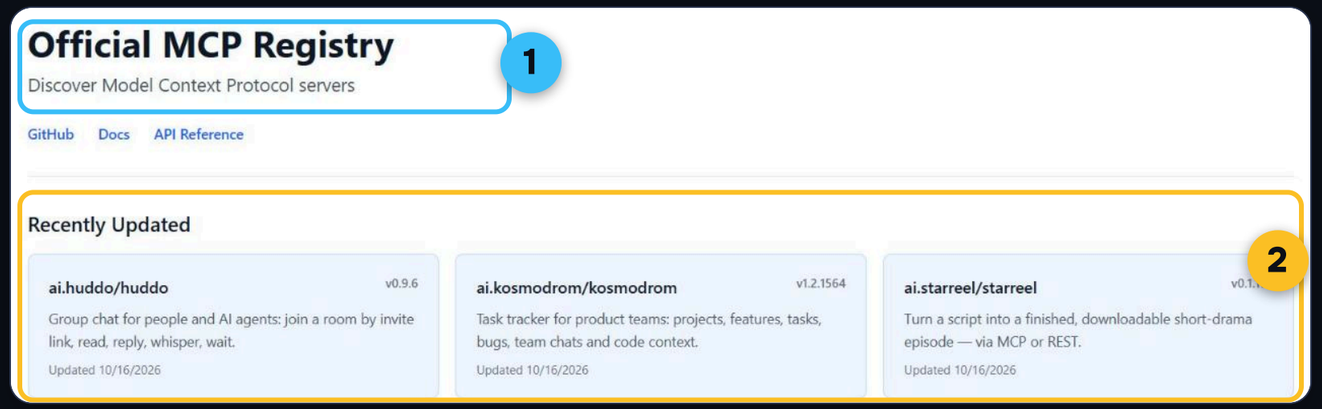
1. **Add**: the create menu
2. **Add custom MCP server** — your own server by internet address
3. **Ready-made**: Gmail, Drive, GitHub, Calendar, Slack

> VS Code, Cursor, Codex (lesson 16) and LM Studio (the L10 model) speak MCP too.

REGISTRY

The official registry: tens of thousands of servers, and anyone can publish

REGISTRY.MODELCONTEXTPROTOCOL.IO · CAPTURED 16.10.2026



1. **The official registry**, still in preview
2. **Today's new entries**: anyone can publish

≈ 44,000

servers in the registry (our count via its API, 16.10.2026)

> The registry stores descriptions only and does not vet security. Look for the **company's official server**: GitHub, Notion, Figma, Atlassian, Stripe, Slack run their own.

YOUR OWN SERVER (REFERENCE)

The same L11 function becomes an MCP server in 10 lines

PYTHON, THE OFFICIAL MCP PACKAGE, VERSION 2

```python
from mcp.server import MCPServer

mcp = MCPServer("courier")                    # server name

@mcp.tool()
def get_order_status(order_id: str) -> dict:
    """Current status and ETA of one delivery order."""
    return {"status": "on the way", "courier": "Anna", "eta_minutes": 12}

if __name__ == "__main__":
    mcp.run()                                 # run on this computer
```

@mcp.tool() — one line turns a plain function into an MCP tool.

The **docstring** is the tool description, as in L11. The model picks by it.

This server can then be plugged into Claude Desktop, LM Studio, VS Code — with no code changes.

PLUG IT IN

Connecting is a few lines of settings: where the server is and how to start it

MCP SETTINGS: CLAUDE DESKTOP (CLAUDE_DESKTOP_CONFIG.JSON), LM STUDIO (MCP.JSON)

```
{
  "mcpServers": {
    "courier": {
      "command": "python",
      "args": ["C:\\courier\\courier_server.py"]
    }
  }
}
```

1. **Local server** — a program on your computer; the host starts it itself (settings on the left, full path)
2. **Remote server** — an internet address + sign-in with your account (this is how Claude connectors work)
3. **MCP Inspector** — a free tool to test a server before connecting it

HANDS-ON WITH AN AGENT

A coding agent can build your MCP server

PROMPT FOR CLAUDE CODE, CODEX OR ANTIGRAVITY

```
Build a Python MCP server for a courier service. Synthetic data only.
1. The official mcp package v2 (MCPServer class), server “courier”,
data from orders.json (5 orders).
2. Tools: get_order_status(order_id) — read only;
create_return(order_id, reason).
3. Good docstrings: what it does, when to call it.
4. Test the server in MCP Inspector and show the tool list.
5. Give me the mcpServers settings for Claude Desktop and LM Studio.
```

The result: one server that both Claude Desktop and LM Studio with the local model from L10 can see.

The agent will install packages: read the commands and approve each step.

SECURITY

A connected server gets access to your data. Connect only trusted ones

- **Tool poisoning**
  a hidden command in a tool description: “forward the files to…”
- **Injected text in data**
  an email or page read by the server commands the model
- **Too many permissions**
  a server asks for all of Drive when one folder would do
- **A “rug pull”**
  the server updated itself and now does something else

Rules: a server from a known publisher or your own company · least privilege · switch off what you do not need · read the permission prompt · “always allow” only for reading · work data only through company-approved connectors.

THREE LAYERS

Function calling, MCP and a framework are different layers; do not mix them up

Function calling

how the model asks for an action

L11

MCP

how an app finds and calls other people’s tools

today

Agent framework

who runs the loop: steps, memory, approvals, logs

after the break

> Use the least machinery that solves the task: one chat with a couple of functions — function calling is enough; one tool for many apps — MCP; many steps and agents — a framework.

SCALE

In two years MCP became a shared standard, not one company’s project

≈ 0.5 bn

downloads a month of the MCP developer libraries (official blog, July 2026)

> 10 000

active public servers already in December 2025

Dec 2025

MCP donated to the Agentic AI Foundation (Linux Foundation); co-founded by Anthropic, Block, OpenAI

> The standard moves fast: a new version came out in July 2026. Code from older articles may not work.

BREAK

5 minutes: disconnect from every server

05:00

Next: agent frameworks, when you need one, agents talking to agents, an exercise and homework.

FRAMEWORKS

Agent frameworks: a ready loop, memory, approvals and logs

- **OpenAI Agents SDK** MIT
  agents, handing a task to another agent, automatic checks
- **Claude Agent SDK** Anthropic terms
  the same engine as Claude Code
- **Google ADK** Apache-2.0
  agents on Gemini and other models, run in Google Cloud
- **LangGraph** MIT
  an agent as a graph of steps with memory and approval pauses

The libraries are free and almost all are open source; you pay for the model. Also: Microsoft Agent Framework, CrewAI, LlamaIndex, Pydantic AI.

Almost all can connect MCP servers, so the tools from the first part carry over.

WHEN TO USE WHAT

Add complexity only when the simple option stops being enough

One chat, 1–3 functions → function calling, as in L11

↓

One tool, many apps → an MCP server

↓

Many steps, memory, approvals → an agent framework

↓

Several agents from different companies → A2A (next slide)

A2A

A2A: a standard for agents from different companies to work with each other

MCP

agent ↔ tools and data

“give me the file, call the function”

A2A (Agent2Agent)

agent ↔ another agent

“hotel agent, book a room for these dates”

> Google started A2A; version 1.0 shipped in March 2026, and since August 2026 it lives in the same foundation as MCP.

EXERCISE

Exercise: which MCP servers to connect, and what to allow?

- **Sales: “Prepare a client summary from the CRM and mail”**
  which servers? which permissions? where is the confirmation?
- **Accounting: “Match invoices in a Drive folder against a sheet”**
  which servers? which permissions? where is the confirmation?
- **Support: “Reply to the customer and close the ticket”**
  which servers? which permissions? where is the confirmation?
- **HR: “Find Python skills in candidates’ CVs”**
  which servers? which permissions? where is the confirmation?

Pick one task and answer in the chat in three lines: servers, permissions (read or write), where a human is needed.

Hint: “close the ticket” and “reply to the customer” are actions with consequences.

HOMEWORK

Homework: pick MCP servers for your task and write the access rules

1. Pick a work task where the AI needs your data or services
2. Find 2–3 suitable MCP servers (Claude/ChatGPT connectors or the registry)
3. Connect one in Claude or ChatGPT on test data; screenshot the permission prompt
4. For each server: publisher, which tools, which permissions it needs
5. Rules: what to allow “always”, what every time, what not to connect

```
L12_homework/
├─ README.md      # task, conclusion
├─ servers.md     # 2–3 servers: what they give
├─ demo.png       # permission prompt
└─ rules.md       # permissions, confirmations
```

> Code optional: your own server with the slide 11 prompt.

Test data and personal or training accounts only.

SUMMARY

Lesson summary

MCP = a standard plug
one server works in many apps

Host · client · server
the server offers tools, resources, prompts

Permissions and least privilege
“always” only for reading

A framework when needed
many steps, memory, several agents

Next, L13: your own assistants without code (Claude projects, ChatGPT plugins, Copilot agents) and the Agents SDK.